

Supplemental Material: Physics-Exact Credit Assignment for Multi-Agent Flutter Suppression

This document collects material referenced from the main text but moved here to meet the journal’s page-length guidance. Section, table, and figure numbers below are internal to this document (prefixed S) and are cross-referenced from the main text by name, not by number. All numerical results here were computed by the same code and are covered by the same seed-count and statistical-correction standard stated where noted in the main text.

S1. Training hyperparameters

Table 1 lists the PPO and network hyperparameters shared across every learned variant in the credit and architecture ablations and the exploration-scale sweep, except where a hyperparameter is itself the subject of an ablation (the initial exploration standard deviation, and the presence or absence of the architectural mechanisms). All runs share parameters across agents, with the agent index supplied as a one-hot input, so the parameter count does not grow with the number of surfaces.

Table 1: PPO and network hyperparameters.

Hyperparameter	Value
Parallel environments	128
Rollout length	128 steps
Epochs per update	4
Minibatches per epoch	4
BPTT segment length	32 steps
Learning rate (Adam)	3×10^{-4}
Discount γ	0.995
GAE λ	0.95
PPO clip range	0.2
Value loss coefficient	0.5
Entropy coefficient	2×10^{-3} (0 in the exploration sweep)
Max gradient norm	0.5
Curriculum fraction	0.4 of training
Hidden layer width (trunk)	128
Phasor encoder hidden width	64
Retained modes per phasor	2
Consensus rounds	2
Action-head output-layer gain (init.)	0.01

S2. Compute and reproducibility

Every learned variant in this paper trains for 1.5×10^6 environment steps across 128 vectorised environments (Table 1) on a single NVIDIA RTX 5000 Ada (laptop) GPU, using PyTorch 2.6. Across the 86 training runs behind the main results and ablations, median wall-clock training time is 331 s (minimum 246 s for a plain policy, maximum 1115 s for the full **mocca** architecture, whose GRU encoder and consensus rounds are the most expensive forward pass in the ablation set), at a median throughput of 4.5×10^3 environment steps per second — the batched environment rather than the network is the bottleneck at this scale, since the plant assembly is vectorised across environments but not across agents. The full set of runs reported in this paper — the credit and architecture ablations, the eight-surface scaling probe, and every additional probe in Section S3 below — totals 12.7 h of GPU time; no result in this paper required infrastructure beyond a single workstation GPU.

S3. Additional probes

The results in the main text are held to a fixed standard: six seeds, Welch’s t -test, Cohen’s d and Hedges’ g , Holm-Bonferroni corrected. Three further questions arose directly out of those results and were checked, but only at one or two seeds each — not because the questions matter less, but because reaching the main standard for every question this study raised would have multiplied its cost several times over. We report them as probes rather than findings: Table 2 gives every seed individually rather than a mean, so that no reader mistakes a two-point spread for an estimated standard deviation.

Table 2: Additional probes, one or two seeds each, at $\sigma = 0.05$. Every healthy-decay value is listed per seed, not averaged, because two points do not estimate a standard deviation. -- marks a probe not run for that variant.

Probe	Variant	Healthy decay [1/s], per seed	Held
<i>Do the negative mechanisms of the architecture ablation also hurt as a residual correction?</i>			
residual + raw comm.	residual_comm	−0.697, −0.247	6/9, 6/9
residual + health-channel consensus	residual_health	−0.518, −0.066	5/9, 5/9
<i>Does supervising the encoder on the true modal phasor help?</i>			
supervised encoder, plain head	aux_encoder	−0.219, −0.029	5/9, 5/9
supervised encoder, residual	aux_residual	−0.431, −0.510	5/9, 5/9
<i>Eight-surface scaling</i>			
consensus, plain head, $N=8$	mocca_nohead	−0.028, −0.015	9/9, 8/9
recurrent only, $N=8$	recurrent_only	+0.135, −0.011	9/9, 9/9

The negative architecture result survives moving to a residual policy The plain-policy architecture ablation found that consensus and raw neighbour sharing hurt. Applying the same two mechanisms to a policy that corrects a decentralised LQG base law instead of replacing it

— the setting where `residual_only` is the strongest result in the paper at -1.53s^{-1} — gives `residual_comm` at -0.697 and -0.247 and `residual_health` at -0.518 and -0.066 : both clearly below `residual_only`, at both seeds. The negative result is not an artefact of the specific plain-policy setting it was first measured in.

Supervising the encoder on ground truth does not help `aux_encoder` adds a loss term supervising the phasor encoder directly on the true modal phasor — privileged information, unavailable outside simulation — rather than letting the encoder’s representation be shaped purely by the RL objective. Both seeds (-0.219 , -0.029) sit below the un-supervised `mocca_nohead` result of -0.308 ± 0.169 , and the residual counterpart `aux_residual` (-0.431 , -0.510) sits far below `residual_only`’s -1.53 . We read this as mild evidence that the credit signal already supplies what the encoder needs to represent, and that an additional supervised loss competes with, rather than complements, the policy gradient here — consistent with the broader pattern that mechanisms motivated by the physics did not survive contact with this specific plant and reward.

Eight surfaces changes what “held” means, not just how many The main text reports that all nine evaluation conditions become feasible for the learned policies at eight surfaces; Table 2 gives the numbers behind that sentence. They deserve a more careful reading than “held” alone provides: `mocca_nohead` holds nine of nine and eight of nine conditions, but at a healthy-condition decay rate of only -0.028 and -0.015 — an order of magnitude weaker than the three-surface result for the same architecture. `recurrent_only` holds every condition at both seeds, yet one seed’s healthy-condition decay is *positive* ($+0.135$). “Held” in Table 2 means the episode did not cross the divergence threshold within its duration, not that the wing was driven to a decisively damped state; a bounded, barely-growing oscillation can hold a condition by this criterion without being a controller we would recommend. We stand by the qualitative claim that eight-surface architectures fail less catastrophically than three-surface ones, and retract any stronger reading of it than that.

S4. Baseline controller synthesis

This section gives the synthesis equations behind every rung of the classical baseline ladder in the main text, so that “discrete-time LQG on local accelerometers” is reproducible from this document alone rather than only from the repository.

Augmented, discretised plant

Actuator lag is part of the design plant, not an unmodelled disturbance added afterwards: with $z = [q, \dot{q}, x_w, \delta]$ (the state of the equation of motion augmented with the deflections) and first-order servo lag of time constant τ_k per surface,

$$\dot{z} = \tilde{A}z + \tilde{B}u, \quad \tilde{A} = \begin{bmatrix} A & B \\ 0 & -\text{diag}(\tau_k^{-1}) \end{bmatrix}, \quad \tilde{B} = \begin{bmatrix} 0 \\ \text{diag}(\tau_k^{-1}) \end{bmatrix}, \quad (1)$$

where A, B are the plant matrices in state-space form and $u \in [-1, 1]^K$ is the normalised command. Every controller in this study is synthesised in discrete time at the 200 Hz control rate via the exact zero-order-hold discretisation

$$\begin{bmatrix} \Phi & \Gamma \\ 0 & I \end{bmatrix} = \exp\left(\begin{bmatrix} \tilde{A} & \tilde{B} \\ 0 & 0 \end{bmatrix} \Delta t\right), \quad (2)$$

giving $z_{t+1} = \Phi z_t + \Gamma u_t$. Designing in continuous time and integrating the resulting gain with a forward-Euler step at $\Delta t = 5$ ms is not a minor approximation here: the wake states have $|\lambda|$ up to $\sim 1.3 \times 10^4 \text{ rad s}^{-1}$, so $\Delta t |\lambda| \approx 63$, and the observer built that way is unstable regardless of how good its gain is.

LQR gain

The state cost weights physical structural energy rather than an arbitrary diagonal, plus a light penalty on held deflection so the optimum does not park surfaces at the travel limit:

$$Q = \begin{bmatrix} K_{\text{modal}} & & \\ & M_{\text{modal}} & \\ & & \text{diag}(w_{\text{rate}}/\delta_{\text{max},k}^2) \end{bmatrix}, \quad R = \text{diag}(w_{\text{effort}}/\delta_{\text{max},k}^2), \quad (3)$$

with $w_{\text{effort}} = 2 \times 10^4$, $w_{\text{rate}} = 10^2$ throughout. The discrete-time algebraic Riccati equation

$$P = \Phi^\top P \Phi - \Phi^\top P \Gamma (R + \Gamma^\top P \Gamma)^{-1} \Gamma^\top P \Phi + Q \quad (4)$$

gives the gain $\mathcal{K} = (R + \Gamma^\top P \Gamma)^{-1} \Gamma^\top P \Phi$ and command $u_t = -\mathcal{K} z_t$ (Anderson and Moore, 2007). **BatchedLQR** solves (4) once at a fixed design speed; **BatchedScheduledLQR** solves it on a 9-point airspeed grid over the evaluation envelope and linearly interpolates $\mathcal{K}$ between grid points; **BatchedJamAwareLQR** additionally zeros the failed surface's column of Γ per failure hypothesis and resolves (4) for each hypothesis on the same grid, so the surviving surfaces' gains already account for the loss rather than compensating for it reactively.

Local rate feedback

The floor of the ladder uses no plant model: with $\dot{h}_k, \dot{\alpha}_k$ the locally sensed plunge and twist rate at surface k 's own station,

$$u_k = -\left(g_h \dot{h}_k + g_\alpha \dot{\alpha}_k\right), \quad g_h = 0.35, \quad g_\alpha = 0.9, \quad (5)$$

clipped to $[-1, 1]$.

Steady-state Kalman observer on local accelerometers

The centralised and decentralised LQG rungs replace the true state z_t in $u_t = -\mathcal{K} \hat{z}_t$ with an estimate driven only by the accelerometer channels the agents themselves observe, $y_t = H z_t$, where H is built from the acceleration rows of the plant rather than a selection matrix, since an accelerometer measures $\ddot{q}$, an algebraic function of the state rather than a state itself. The steady-state predictor-form filter is

$$\Sigma = \Phi \Sigma \Phi^\top + W - \Phi \Sigma H^\top (H \Sigma H^\top + V)^{-1} H \Sigma \Phi^\top, \quad (6)$$

$$L = \Phi \Sigma H^\top (H \Sigma H^\top + V)^{-1}, \quad (7)$$

$$\hat{z}_{t+1} = \Phi \hat{z}_t + \Gamma u_t + L (y_t - H \hat{z}_t), \quad (8)$$

with process noise $W = 10^{-4} I$ and measurement noise $V = 10^{-6} \text{diag}(\|H_i\|^2)$ scaled per-channel, since accelerometer rows of H have norms of order 10^4 – 10^5 and an unscaled V demands impossible

measurement precision and drives L to blow up (Anderson and Moore, 2007). **BatchedLQG** solves (6)–(8) once with H built from all six channels and $\mathcal{K}$ commanding all three surfaces jointly, on the same speed grid as the scheduled LQR. **BatchedDecentralizedLQG** solves an *independent* instance of (6)–(8) per surface k , with H restricted to k ’s own two accelerometer rows and Γ restricted to k ’s own column, so agent k ’s estimate, gain and command never depend on any other surface’s measurement or actuation — the discrete-time, model-based analogue of the information structure the learned policies are given. Its scheduling grid spans the same $[1.0, 1.5] U_f$ range as the training curriculum exactly, by construction rather than coincidence, so the comparison in the main text is not confounded by the classical design being scheduled over a narrower envelope than the learned policy was trained on.

Re-tuning the decentralised LQG for control-authority margin

BatchedRobustDecentralizedLQG is structurally identical to **BatchedDecentralizedLQG**: same per-surface observer, same restriction to each agent’s own two accelerometers and own actuator. Two remedies for its lack of a guaranteed margin (Doyle, 1978) were checked. The first, loop transfer recovery (Doyle and Stein, 1981), boosts the process-noise weighting W in (6) by $\rho b_k b_k^\top$ (agent k ’s own reduced input column) as $\rho \rightarrow \infty$, pushing the filter loop toward the full-state loop shape. Sweeping ρ from 0 to 10^4 changes the control-authority margin (the largest reduction in true control authority, relative to the design assumption, the closed loop remains stable under, found by bisection on the discrete-time closed-loop spectral radius) not at all: it stays at 0.158 at $1.45 U_f$ throughout. Comparing against the same gain applied with perfect (observer-free) state feedback, whose margin is also 0.158, shows why: the observer is not the bottleneck, so there is nothing for an observer-side remedy to recover.

The second remedy, and the one **BatchedRobustDecentralizedLQG** actually uses, re-weights the LQR cost (3) itself: w_{effort} increased from 2×10^4 to 10^6 , with w_{rate} , W and V unchanged. This raises the control-authority margin from 0.158 to 0.179 before plateauing (no further gain to $w_{\text{effort}} = 10^7$), at a cost to nominal decay rate documented in the main text. `scripts/robust_baseline_check.py` reproduces both checks.

References

- Brian D. O. Anderson and John B. Moore. *Optimal Control: Linear Quadratic Methods*. Dover Publications, 2007.
- John C. Doyle. Guaranteed margins for LQG regulators. *IEEE Transactions on Automatic Control*, 23(4):756–757, 1978. doi: 10.1109/TAC.1978.1101812.
- John C. Doyle and Gunter Stein. Multivariable feedback design: Concepts for a classical/modern synthesis. *IEEE Transactions on Automatic Control*, 26(1):4–16, 1981. doi: 10.1109/TAC.1981.1102555.